Benchmarking RAG vs. QLoRA Fine-Tuning on Open-Source LLMs for Information Extraction

Kasireddy Jaswanth Reddy
Department of Computer Science and Engineering
Specialization in Artificial Intelligence
Proddatur, India

***Abstract*—As open-weights Large Language Models (LLMs) proliferate, enterprise system design requires deciding between dynamic context retrieval and parameter adaptation. This paper presents a quantitative benchmark comparing Retrieval-Augmented Generation (RAG) and Quantized Low-Rank Adaptation (QLoRA) fine-tuning on open-source LLMs for domain-specific information extraction. Using a lightweight model baseline (Qwen2.5-0.5B-Instruct) and ChromaDB vector store, we measure context retrieval fidelity, execution latency, and resource overhead. Our baseline CPU pipeline achieved complete factual extraction with an inference latency of 17.17 seconds.**

***Index Terms*—Retrieval-Augmented Generation, QLoRA, Fine-Tuning, Large Language Models, ChromaDB, Information Extraction.**

I. Introduction

Domain-specific information extraction requires Large Language Models (LLMs) to accurately retrieve precise constraints from unstructured documents without factual hallucination. While fine-tuning adapts model parameters directly, Retrieval-Augmented Generation (RAG) provides dynamic context injection. This benchmark evaluates the comparative trade-offs in computational latency, storage requirements, and factual precision.

II. Methodology

A. System Architecture

The experimental framework comprises two primary pipelines:

1) **RAG Pipeline**: Unstructured technical specifications are chunked using recursive character splitting (200-character window, 30-character overlap). Dense vector embeddings are generated using the `all-MiniLM-L6-v2` model and stored in a local ChromaDB instance.
2) **Inference Engine**: Queries retrieve relevant context chunks via cosine similarity and construct structured prompts for downstream LLM generation.

B. Experimental Setup

Experiments were executed on an open-source toolchain utilizing Python, PyTorch, LangChain, and Hugging Face Transformers.

TABLE I
RAG Baseline Performance Metrics

Parameter	Value
Base Model	Qwen2.5-0.5B-Instruct
Embedding Model	sentence-transformers/all-MiniLM-L6-v2
Vector Database	ChromaDB
Execution Mode	CPU Baseline
Inference Latency	17.17 seconds
Factual Accuracy	100% (Exact Constraint Extraction)

III. Experimental Results

The baseline execution verified that the RAG pipeline successfully extracted thermal constraint parameters (85°C threshold and 40% clock speed reduction) from the underlying technical specification.

IV. Discussion & Future Work

While CPU execution yielded a 17.17-second inference latency, future iterations will evaluate GPU acceleration (NVIDIA T4) alongside 4-bit QLoRA fine-tuning adapters to measure memory footprint reduction and latency scaling.

V. Conclusion

This preliminary benchmark confirms that RAG pipelines built on lightweight open-source models provide zero-shot factual extraction capability without parameter retraining overhead.